



Quant in your pocket

NUS Orbital 2026

Milestone 2

[Video](#) [Poster](#) [Log sheet](#)

By O(Alpha)

Zhao Shizhen (A0322526L)

Tan Jia Jun (A0321311A)

Table of Contents

Section	Subsection	Page
Foreword		5
Introduction		6
	Admin	6
	Proof of concept	7
	Setup Guide	7
	Our Aim	8
	Our Motivation	8
	User stories	9
Features		11
	Features Overview	11
	Login	14
	Onboarding	15
	Dashboard	18
	Portfolio	20
	Agent Settings	22
	Activity	24
	Landing Backtest	25
	Software Architecture Diagram	28
UML Diagrams		29
	User Flow Diagram	29
Database Diagram		31

Technical Overview		50
	Single-symbol Kalman filter and state-space model	50
	Moving-average crossover	53
	Execution semantics	54
	Performance and risk metrics	54
	Strategy Catalogs	55
	Parameter reference	56
Architectural Philosophy		58
	Decoupled architecture and separation of concerns	58
	System dataflow path	59
	Core architecture components	59
	Architectural segregation	60
	Design patterns: DRY and KISS	61
	Concurrency, isolation, and lifecycle safety	61
	Data integrity by construction	62
Quality Assurance		63
	Quality Control Stages	63
	Continuous Integration	63
	Continuous Deployment	63
	Unit Testing	64
	User Testing	71

Milestone Planning	72
Future Roadmap	74
Monetization Strategy	74
Final Reflection	75

Foreword

Dear Reader,

This report serves as an open window into O(Alpha), chronicling our entire journey from an initial spark of an idea to a functional execution, alongside the engineering philosophy that drove our development.

To give you a transparent look at how the system builds and scales under the hood, we have included comprehensive application mockups, technical diagrams, and detailed architectural documentation.

As you dive in, please note that this is our current Proof of Concept (POC). The baseline styling, aesthetics, and user interface are still evolving, meaning the live app will look quite different from the finalized design concepts shown here. This phase is entirely about proving the core mechanics and robustness of our engine.

We warmly invite you to explore O(Alpha), push its boundaries, and share your candid feedback with us. Your insights are what will help us iterate faster and take this product to greater heights.

Thank you for being a part of our journey.

Warm regards,

Shizhen & Jiajun

Introduction

Admin

Team name: O(Alpha) (Team 6645)

Proposed level of achievement: Apollo

Overview

O(Alpha) is a high-performance quantitative trading and portfolio execution platform designed to automate market analysis, strategy generation, and trade execution through mathematical modeling and predictive algorithms.

The platform integrates continuous market scanning, statistical filtering, and regime-aware execution into a unified system capable of processing live asset streams in real time. By reducing reliance on emotional decision-making and manual chart interpretation, O(Alpha) aims to provide users with a more systematic and data-driven trading experience.

The system architecture is built around a decoupled infrastructure:

- **Backend Execution Engine (Go):** Handles high-speed computation, signal generation, portfolio simulation, and execution pipelines.
- **Frontend Terminal Interface:** Focuses on financial visualization, real-time telemetry, and intuitive controls.

O(Alpha) currently supports multiple quantitative strategy frameworks across two execution surfaces:

- **Single-symbol Kalman Filter**
Uses predictive filtering and statistical deviation analysis for the landing-page backtest and legacy symbol-agent path.
- **Single-symbol Moving Average Crossover**
Detects trend-following crossover events and remains the default base strategy for several educational and ML-meta-label flows.
HMM Ensemble / Regime Overlay
API-exposed single-symbol agent mode that combines MA and Kalman votes while using an HMM risk overlay for regime state and sizing context.

- **ML Meta-Label Strategy**

API-exposed wrapper that loads a model artifact, builds features and calibrates probabilities.

- **Portfolio Catalog Strategies**

Primary paper-trading path built on multi-symbol PortfolioStrategy implementations: LGBM h63 ranker sleeves, deterministic h63 proxy sleeves, low-volatility diagnostics, ranked sleeves, benchmark-funded TSMOM, and composite momentum challengers.

The platform continuously processes live price streams and transforms them into systematic trading signals through automated mathematical pipelines.

Proof of concept

Video

<https://drive.google.com/file/d/1cLmldqooo7drd9C3rpafV5F6Xso6HH80/view?usp=sharing>

Deployment

Frontend

App dashboard can be accessed via

Login -> "Demo Onboarding" -> app dashboard

<https://o-alpha-tan.vercel.app/>

Backend

<https://o-alpha.onrender.com/>

Our Aim

O(Alpha) aims to provide an accessible, intelligent, and automated trading platform that allows both beginners and experienced traders to participate in financial markets with greater confidence and reduced emotional bias.

By combining quantitative finance, predictive analytics, and real-time execution systems, we seek to simplify trading into a seamless and intuitive experience.

Our Motivation

Modern trading environments are increasingly complex and data intensive. Many retail traders struggle with:

- Information overload from rapidly changing markets
- Emotional decision-making during volatile conditions
- Difficulty identifying reliable trading signals
- Poor risk management and capital allocation

We realized that most existing trading platforms either:

- Require deep financial knowledge and manual analysis, or
- Provide simplistic tools that fail under changing market conditions

As a result, many individuals entering the trading world are unable to make informed and disciplined decisions.

O(Alpha) was created to bridge this gap by automating signal processing, filtering market noise, and adapting strategies dynamically based on changing market regimes. Our goal is to reduce cognitive burden while enabling smarter, more consistent portfolio execution through quantitative systems.

User stories

User	User Story Statement	Priority	Technical Implementation
Systemic Trader	As a systemic trader, I want a minimalist, modern terminal workspace so that I can monitor real-time statistical metrics without interface distractions.	Essential	Dark-theme terminal layout with numerical values in a monospace font for scanability (see Dashboard, p.18). Chosen over a light/card-based UI to reduce eye strain during multi-hour monitoring, at the cost of a steeper initial learning curve for new users.
Statistical Analyst	As a statistical analyst, I want to toggle between mean-reversion and momentum engines instantly to test distinct mathematical strategies.	Essential	A toggle component (StrategySelector) swaps the visible input fields and the strategy_type field in the BacktestRequest payload between KALMAN and MA_CROSSOVER (see Landing Backtest, p.25, and Parameter Reference, p.56).

Quant Engineer	As a quant engineer, I want precise control over process noise variables to fine-tune how fast the estimation engine tracks true asset values.	Typical	Debounced range sliders that update local state immediately (no perceptible input lag), but only commit to the backend on release, at 0.001 step precision for q_noise (see Technical Overview §Kalman filter, p.50).
Risk Manager	As a risk manager, I want to set market noise rejection thresholds (\$R\$) to ignore random price spikes during periods of high volatility.	Typical	Dynamic track-filled slider mapping adjustments directly to the measurement noise parameters of the state-space equations.
Execution Trader	As an execution trader, I want to define trade triggers using precise standard deviation boundaries to protect entries.	Typical	Tactile range inputs displaying execution boundaries on a mono-spaced layout, with active compression feedback on user press.
Fund Allocator	As a fund allocator, I want to view dynamic YTD cumulative P&L metrics so that I can evaluate strategy performance across active fiscal cycles.	Essential	Sliced timeframe arrays updating large numerical displays in real time via custom client-side data aggregation wrappers.

Risk Auditor	As a risk auditor, I want a clear visualization of real-time Sharpe and Sortino Ratios to confirm our active strategies stay within optimal portfolio parameters.	Typical	Sharpe/Sortino cards are visually separated from other metrics using a distinct border style, so a risk auditor can locate them without reading labels (see Portfolio, p.20).
Compliance Officer	As a compliance officer, I want a visual tracking tool for Maximum Drawdown to ensure our running simulations don't cross our hard risk limits.	Essential	Sleek linear progress metrics mapped to safe surface container tokens, tracking current drawdown states down to the pixel.
Research Auditor	As a research auditor, I want each strategy to show paper-only status and evidence paths so that I can tell what is validated and what is only experimental.	Essential	GET /api/v1/strategies/catalog returns deployment_status, promoted_checkpoint, evidence_paths, paper_only, and requires_model_artifacts metadata.
Systems Engineer	As a systems engineer, I want the client app to send typed, validated request payloads to our Go backend to prevent engine compilation and runtime failures.	Essential	Shared TypeScript interface structures connecting the front-end parameters directly to the backend API schemas.

Features

S/N	Feature	Milestone	Brief Explanation
Core Features			
User-Facing Application Screens			
1	Back Testing	1	Allows trial users to perform back testing directly on our landing page
2	Login	1	Authentication for logging in (for returning users) and sign up (for new users)
3	Onboarding	1	First-time authenticated users configure their risk profile before using the dashboard
4	Dashboard	1	Dashboards for portfolio value, agent controls, allocations, trade logs.
5	Agent Settings	1	View and edit trading agent risk parameters after onboarding
6	Portfolio	1	Detailed holdings view positions, allocation, historical portfolio value

7	Activity Logs	1	Audit trail of executions and system alerts.
Quantitative Strategy Engines			
8	Paper Trading Engine	2	Live background agent workers that execute simulated trades against real-time market data
9	Hidden Markov Model (HMM) Regime	2	Three-state, nine-symbol forward-filter HMM that classifies market regime as Low-Vol Trend, Medium, or High-Vol Stress
10	HMM Risk Overlay & Safety Triggers	2	Regime-aware exposure multiplier with hysteresis, crisis de-risk, realized-vol cap, and drawdown guard
11	HMM Ensemble Decision Layer	2	Regime-weighted MA + Kalman ensemble that adjusts signal weights and position sizing by market regime
12	Pairs Trading Engine (Kalman Cointegration)	2	Two-state Kalman hedge filter tracking cointegration spread, with Z-score entry/exit, stop-loss quarantine, and periodic pair retesting
13	ML Meta-Label Strategy	2	Machine-learning gating layer that filters base-strategy buy signals through a probability model with 47-feature vector and calibration
14	Portfolio Backtest Engine	2	Multi-symbol portfolio simulator with cost model (spread, slippage, commission), aligned bar panels, and per-sleeve risk overlay
15	Strategy Catalog	2	Nine production-grade portfolio strategies (LGBM ranker, deterministic ranker proxy, composite momentum) organized by risk profile with promotion gating

16	Artifact-aware Ranker Runtime	2	LGBM ranker strategies read local model artifacts from OALPHA_DAILY_RANKER_ARTIFACT_ROOT and fail closed when artifacts are absent.
Execution & Data Infrastructure			
17	Websocket Market Data Streaming	2	Real-time Alpaca WebSocket connector with auto-reconnect for low-latency bar delivery to live agents
18	Portfolio Agent Manager	2	Multi-symbol portfolio worker orchestration with concurrent execution, per-sleeve telemetry, and graceful lifecycle management
19	Enhanced Database Schema	2	Six new tables (universes, universe_members, portfolio_backtest_runs, sleeve_returns, ml_model_artifacts, pair_candidates, strategy_trials) and multi-dataset bar support
20	Backtest Streaming Endpoint	2	Server-Sent Events backtest endpoint for real time rendering of the simulation run.
21	Agent List & Strategy Catalog APIs	2	New endpoints to enumerate running agents and browse available portfolio strategies
22	DB-backed Portfolio Execution Router	2	Converts target weights into idempotent paper fills, cash ledger updates, positions, portfolio snapshots, and rebalance/risk alerts.
23	Portfolio Live State Stream	2	Streams snapshots and latest price marks to keep dashboard summary, allocation, and history synchronized with backend state.

Login

The login system is designed around two core security principles: stateless authentication and zero plaintext credential storage.

Log In or Create Account

Use an existing username to sign in, or enter a new one to create your O(Alpha) account.

Username

Password 

☐ Remember Me [Reset via support](#)

Start

Passwords are never stored in a readable form. On registration, bcrypt is used to hash credentials with a cost factor that makes brute-force attacks computationally expensive. Even if the database is compromised, raw passwords cannot be recovered. On login, the submitted password is compared against the stored hash, never against a stored plaintext value.

Authentication state is managed through a stateless JSON Web Token (JWT) signed with HMAC-SHA256. This eliminates the need for server-side session storage: the backend does not need to maintain a session table or share session state across instances, which means the system scales horizontally without session-synchronization complexity. The token is verified on every protected request purely by signature validation, if the signature is invalid or the token is expired, the request is rejected.

The client persists this token and attaches it to all subsequent calls to protected routes (`/api/v1/*`), so every authenticated action is traceable to a verified identity without a database lookup per request.

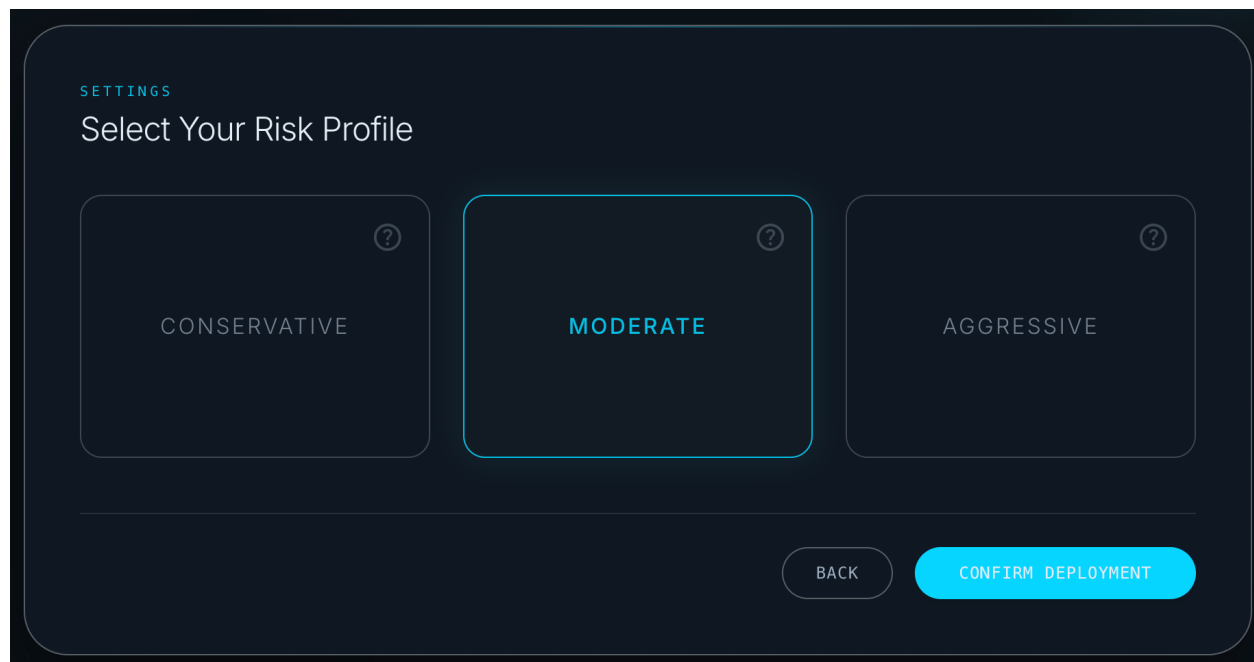
Login and registration are unified into a single action to reduce user drop-off. The backend checks whether the submitted username exists. If it does, it validates credentials and issues a token. If it does not, it automatically registers the account, hashes the credential, and issues a token in the same response. The onboarding state is initialized to "False" so the system can route new users into the setup wizard on their first session.

Onboarding

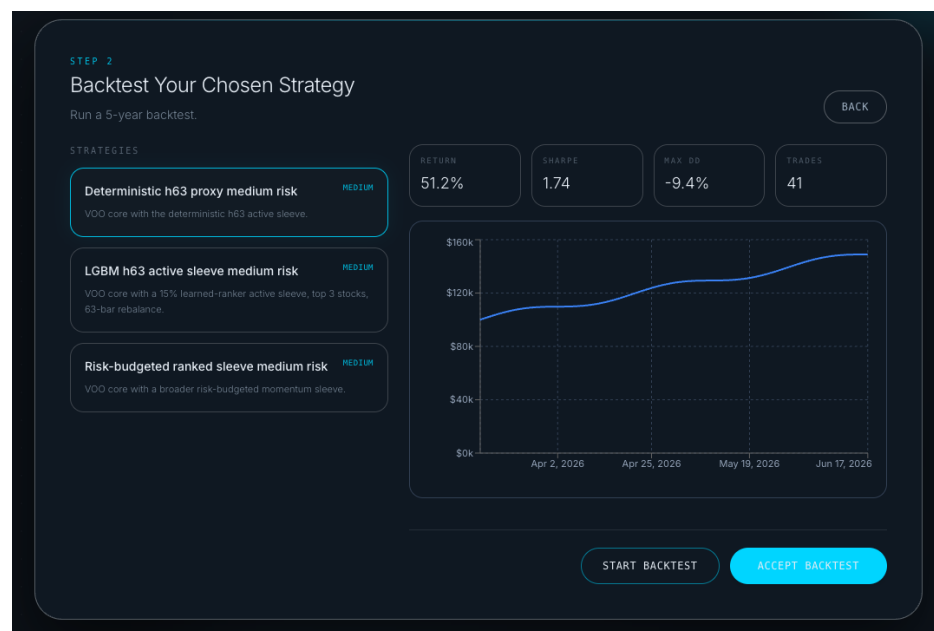
First-time authenticated users begin setup, choose a risk profile, review a matching paper-only catalog strategy, run a five-year portfolio catalog backtest, and accept the result before the dashboard is unlocked.

The first screen selects the risk profile and the following screen loads `/api/v1/strategies/catalog`, filters strategies by the selected risk bucket, shows the available strategy cards, and renders return, Sharpe, max drawdown, trades, and the equity curve for the chosen strategy.

Start Backtest calls `POST /api/v1/backtest/stream` with a `PORTFOLIO_CATALOG` payload, the full default universe, the chosen `strategy_key`, and risk-derived settings. Accept Backtest remains disabled until a completed result exists; accepting saves `/api/v1/user/settings` and then calls `/api/v1/user/onboarding/complete` with `backtest_accepted=true`.



The second onboarding screen, shown below, is the acceptance gate. The user can compare catalog strategy cards, inspect the generated metrics and equity curve, rerun the test, or accept the backtest to persist their configuration.



Technicalities and Motivations

Frontend	
Topic	Explanation
Reason for necessary fields	To drastically lower user acquisition friction, we eliminated traditional separated signup flows. The single form requires only a Username and a Password. The risk profile collected during onboarding is mandatory because the backend cannot safely instantiate a portfolio strategy or run a backtest without a bounding risk parameter: it governs leverage limits, position caps, and stop-loss margins. Collecting it post-login rather than at registration keeps the sign-up barrier low while still enforcing the constraint before any capital-at-risk operation.
Onboarding UI flow	The wizard is now a deliberate three-step flow: welcome, risk-profile selection, and catalog backtest acceptance. Risk cards select only the profile; an explicit Next button advances to the backtest screen.
Catalog strategy cards	The frontend loads <code>/api/v1/strategies/catalog</code> , filters strategies by the selected low/medium/high risk bucket, and renders selectable strategy cards with display name, risk label, and description.
Error and disabled states	State flags disable Back, strategy selection, Start Backtest, and Accept Backtest while the catalog is loading, the backtest is running, or settings are being saved. Backend errors render inline instead of silently advancing the user.

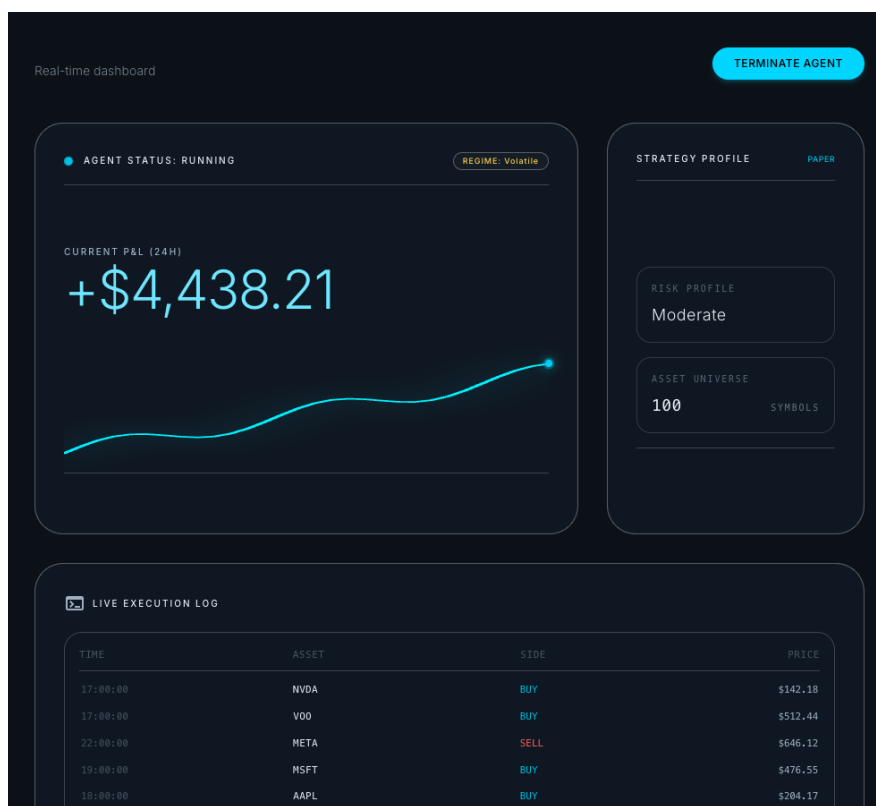
Backend	
Topic	Explanation
Unified Authentication Handler	When the frontend hits the authentication endpoint, the controller logic first executes an existence check on the provided unique username. If the username does not exist, the backend seamlessly triggers a sign-up mechanism: it hashes the password, instantiates a new user document profile with default parameters, and flags the user record with <code>onboarded: false</code>. It then immediately signs the user in to start the onboarding, bypassing the need for a separate registration step.
Catalog and risk validation	GET <code>/api/v1/strategies/catalog</code> returns the default universe, recommended risk-profile map, and StrategySpec metadata.

	Onboarding uses this to ensure the selected strategy belongs to the chosen risk bucket.
Backtest stream gate	POST /api/v1/backtest/stream accepts portfolio strategies only. For onboarding it receives strategy_type=PORTFOLIO_CATALOG, symbols=default_universe, timeframe=1Day, initial_cash=100000, and parameters.strategy_key, then emits started/progress/completed/error NDJSON events.

Dashboard

The dashboard is the authenticated command centre for paper trading. It surfaces the current portfolio P&L, active agent status, regime label, recent fills, allocation, and a simplified strategy profile card that only displays the user-facing risk profile and the selected asset universe size.

The launch button starts or terminates the one-user-one-agent portfolio flow. Once active, the screen hydrates from backend agent state, portfolio snapshots, positions, trades, and the live portfolio stream so values change when paper fills or price marks are persisted.



Technicalities and Motivations

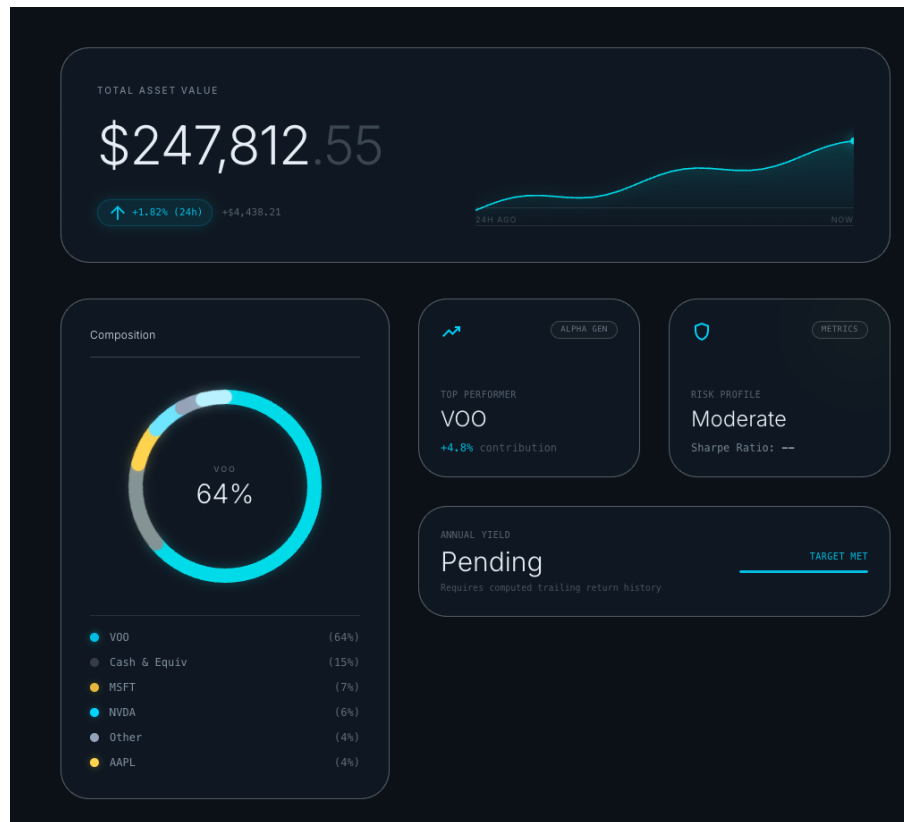
Frontend	
Topic	Explanation
Screen composition	The page composes BalanceCard, StrategyControls, ExecutionLog, and PortfolioAllocation inside AppShell. The strategy card is display-only and intentionally exposes only Risk Profile and Asset Universe.
State hydration	SWR loads /api/v1/user/portfolio/summary, /history, /positions, /trades, /strategies/catalog, and /agent/list. The active/idle state is derived from the backend agent list, not local component state.
Live updates	streamPortfolioLive patches summary, positions, and history in-place. Price events are applied through portfolioLiveState helpers so dashboard values and allocation can update without a full page refresh.
Agent control UX	The Launch/Terminate button calls portfolioAgentApi.start or stop, disables during synchronization, and renders backend errors in an inline alert panel.
Backend	
Topic	Explanation
Agent state	GET /api/v1/agent/list returns active agent_runs with strategy_type PORTFOLIO_CATALOG, strategy_key, heartbeat, and runtime_state such as the synced regime label.
Portfolio state	Portfolio endpoints read canonical DB snapshots, positions, and fills. GetPortfolioSummary, GetPortfolioHistory, GetActivePositions, and GetExecutionStream all scope records to the authenticated user.

Launch/stop flow	POST /api/v1/agent/portfolio/start validates risk/profile strategy alignment, creates an agent_runs row, starts the portfolio orchestrator, and marks the run as running. Stop marks the active run stopped and writes an alert.
Streaming	GET /api/v1/user/portfolio/live emits NDJSON snapshots and price events. With Alpaca credentials, latest prices are marked into positions before snapshots are saved.

Portfolio

The portfolio screen expands the dashboard allocation into a holdings-centric view. Users can inspect total asset value, 24-hour change, composition, top performer, risk profile, target progress, and the live positions table in one place.

Unlike the marketing performance page, this screen is tied to persisted paper-trading state: positions, cash, current prices, unrealized P&L, and portfolio history come from the backend repository and are refreshed through the same live stream used by the dashboard.



Technicalities and Motivations

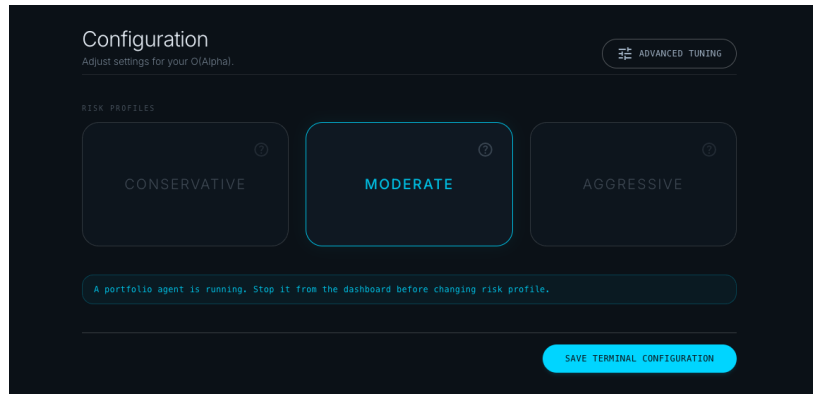
Frontend	
Topic	Explanation
Summary and charting	The page reads portfolio summary and history through SWR, then renders the total value, 24-hour change pill, and sparkline from chronological snapshot data.
Composition ring	Current positions are ranked by exposure, grouped into visible slices plus cash/other, normalized to percentages, and rendered as a donut allocation chart.
Live position table	The positions table uses server positions when authenticated and applies live price events to flash rows up or down when a quote changes.

Export and display controls	Users can expand beyond the default visible positions and export the current position set to CSV using the shared downloadCSV helper.
Backend	
Topic	Explanation
Snapshot source	PortfolioRepository.SavePortfolioSnapshot computes cash, positions value, total asset value, realized/unrealized P&L, 24-hour change, and target progress from DB state.
Position source	GetActivePositions returns open long positions with qty, avg_entry_price, current_price, unrealized_pnl, and exposure for the authenticated user.
Price marking	refreshPortfolioMarks fetches latest Alpaca prices for open symbols, calls MarkPositionPrice, and records a fresh snapshot before serving portfolio reads.
Live stream	StreamPortfolioLive emits the latest summary, positions, and history first, then continues with Alpaca trade/price events or a timed snapshot fallback.

Agent Settings

Agent Settings lets the user manage risk posture and paper-agent constraints after onboarding. The visible cards map the user-friendly profiles to backend risk buckets, while the advanced controls store leverage, maximum positions, stop-loss, take-profit, and rebalance cadence.

Risk changes are deliberately gated. If a portfolio agent is running, the frontend blocks profile changes and tells the user to stop the agent first. When a risk profile changes, the user must run and accept a catalog backtest before the settings payload can be saved.



Technicalities and Motivations

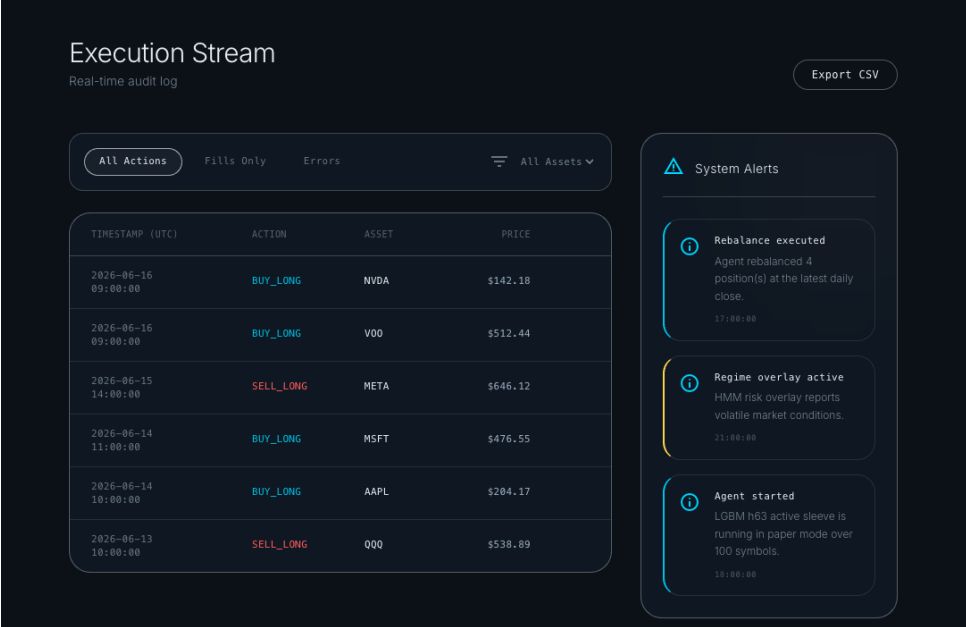
Frontend	
Topic	Explanation
Risk profile cards	The three profile cards use local state for selection and flip-state details, then compare the selected profile against the saved profile to determine whether validation is required.
Catalog strategy selection	The screen loads <code>/api/v1/strategies/catalog</code> , filters strategies by the chosen risk bucket, and defaults to the matching recommended catalog key.
Backtest validation	<code>runBacktestStream</code> powers the validation panel, progressively rendering equity points while the backend emits NDJSON progress events. Accept Backtest records the strategy key that passed validation.
Save gating	The Save button is disabled while backtesting/saving and when a changed profile has not been accepted, or when <code>/agent/list</code> reports a running portfolio agent.
Backend	
Topic	Explanation
Settings read/write	GET <code>/api/v1/user/settings</code> returns saved <code>agent_settings</code> ; POST <code>/api/v1/user/settings</code> validates bounds before upserting <code>risk_profile</code> ,

	leverage, max_positions, stop_loss_pct, take_profit_pct, and rebalance_freq.
Running-agent guard	When risk_profile changes, SaveUserSettings checks ListActiveAgentRuns and rejects the update until the active portfolio agent is stopped.
Backtest acceptance guard	Risk-profile changes require backtest_accepted=true and a strategy_key whose catalog risk bucket matches the requested profile.
Runtime consumption	Portfolio runtime settings are converted into max gross exposure, active-position caps, stop-loss/take-profit exits, and rebalance eligibility inside the portfolio worker path.

Activity

The Activity screen is the audit console for the paper-trading system. It shows recent persisted execution records beside system alerts so a user can understand what the agent did and why it did it.

The interface supports action filters, asset filters, trade-limit selection, and CSV export. This makes the screen useful both as an end-user history page and as a debugging surface when validating agent behavior.



Technicalities and Motivations

Frontend	
Topic	Explanation
Execution stream table	SWR reads /api/v1/user/portfolio/trades with a configurable limit, then renders timestamp, action, asset, price, size, slippage, and status.
Filtering	The toolbar filters by All Actions, Fills Only, or Errors, and the asset selector is derived from the unique symbols present in the loaded execution stream.
System alerts	The side panel reads /api/v1/user/portfolio/alerts and renders alert type, title, description, and UTC timestamp with visual severity treatments.
CSV export	The current filtered trade rows are exported through downloadCsv as oalpha-activity-log.csv, preserving the audit fields shown on-screen.
Backend	

Topic	Explanation
Trade retrieval	GetExecutionStream reads fills joined to orders and normalizes sides into BUY_LONG, SELL_LONG, SELL_SHORT, or COVER_SHORT before returning the latest rows.
Alert retrieval	GetSystemAlerts returns recent user-scoped alerts from portfolio_alerts with limit clamping to prevent oversized reads.
Alert sources	Portfolio orchestrator and execution routers insert alerts for agent start, stop, resume, rebalance execution, risk exits, and resume failures.
Audit durability	Because activity reads from persisted orders, fills, and alerts, the screen survives refreshes and backend restarts instead of depending on transient frontend logs.

Landing Back Test

The landing-page performance screen lets non-authenticated visitors run a single-symbol back test before signing in. The user enters a symbol, chooses between Kalman Filter and Momentum / MA Crossover, adjusts strategy parameters, and runs the simulation directly from the page.

Returned results populate the real-time metrics section with cumulative P&L, Sharpe ratio, max drawdown, and the equity curve. If the backend cannot run the request, the frontend falls back to a default curve and displays an error note rather than leaving the section blank.

Symbol

AAPL

SIMULATION ENGINE

Kalman Filter

Momentum

Process Noise

0.01

Noise Rejection

0.5

Execution Threshold

± 2

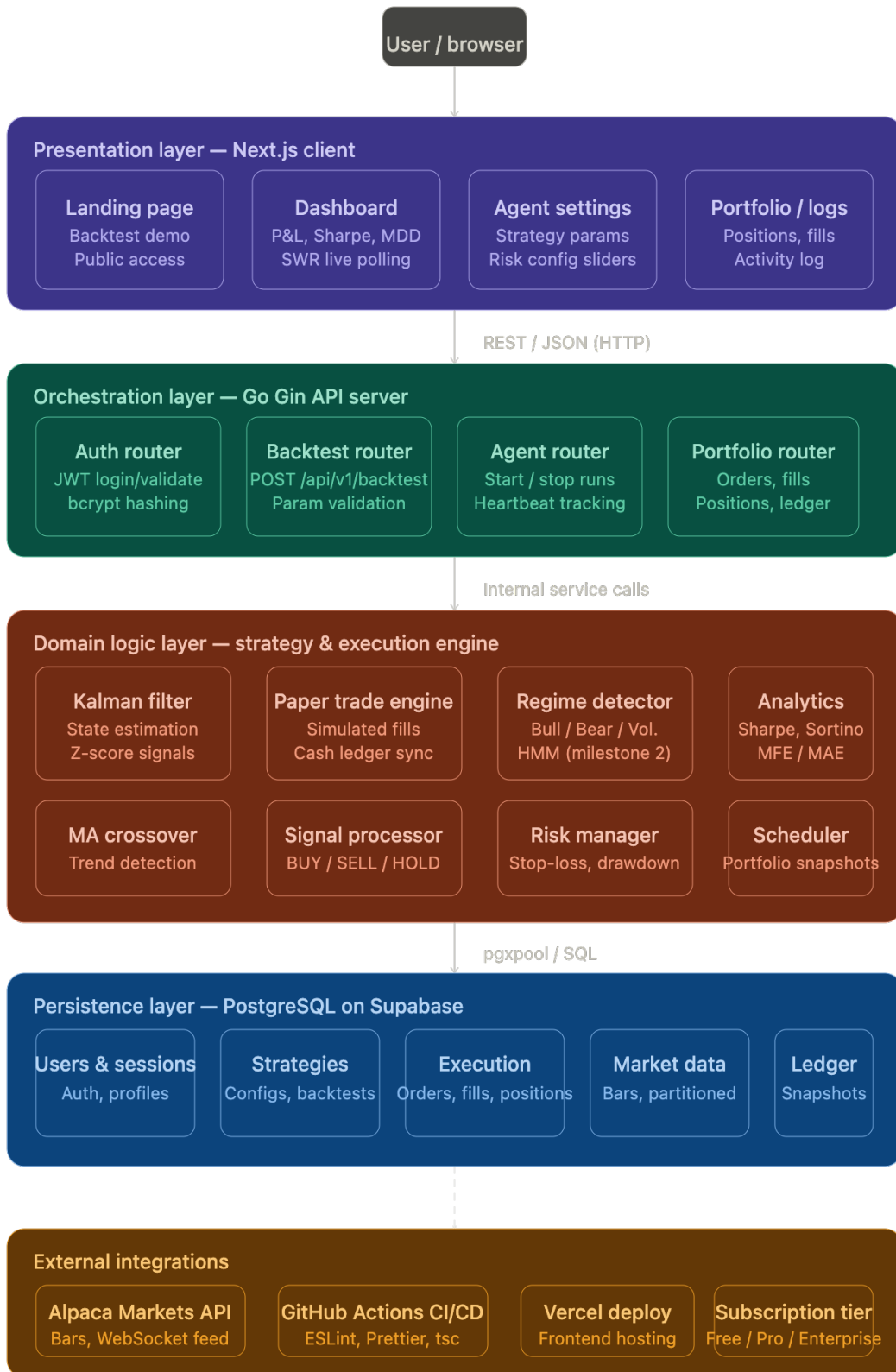
Run Backtest

Technicalities and Motivations

Frontend	
Topic	Explanation
Strategy selector	StrategySelector owns the Kalman vs Momentum toggle and emits a StrategyConfig containing q_noise, r_noise, z_threshold, fast_period, and slow_period.
Backtest request	PerformancePage builds a BacktestRequest using the selected symbol, one-year date range, 1Day timeframe, and selected strategy parameters before calling runBacktest.
Metrics rendering	TelemetryMetrics consumes the returned equity_curve and metrics to render cumulative return, Sharpe ratio, max drawdown, and the charted equity path.
Fallback behavior	If runBacktest fails, the page swaps in DEFAULT_EQUITY_CURVE and explains that fallback performance data is being displayed.
Backend	
Topic	Explanation

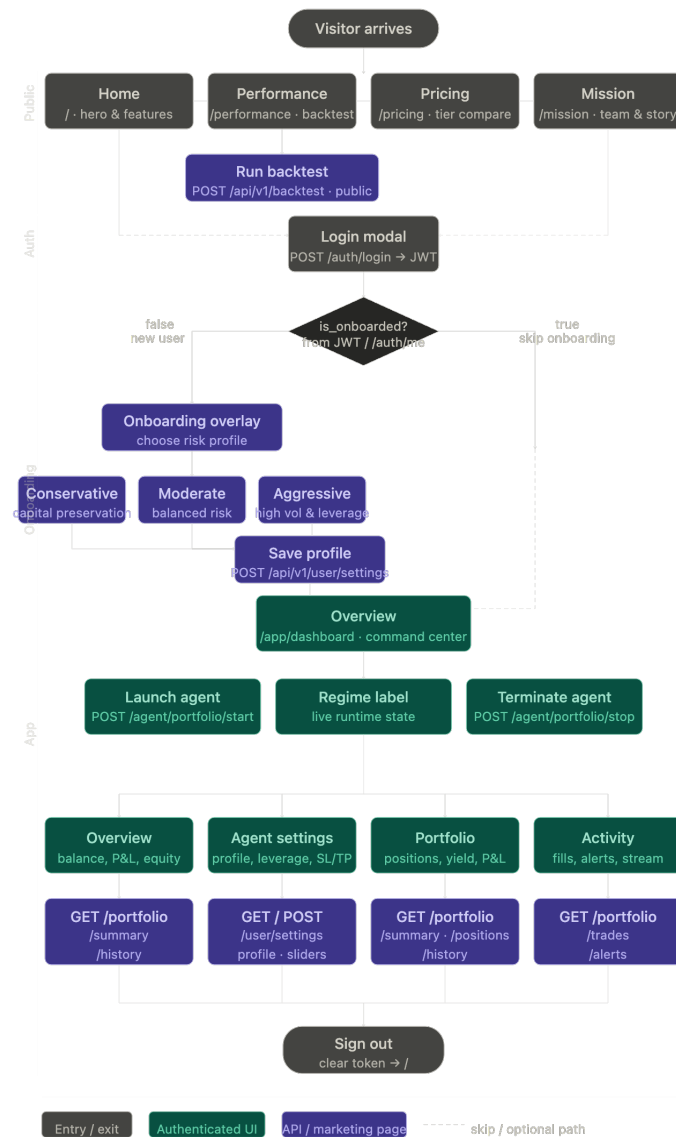
Backtest endpoint	POST /api/v1/backtest parses BacktestRequest, defaults timeframe to 1Day and initial_cash to 100000, and loads bars from the repository for the requested symbol.
MA crossover validation	MA_CROSSOVER defaults to fast=10 and slow=30, rejects fast_period >= slow_period, then runs backtest.NewMACrossoverStrategy.
Kalman validation	KALMAN defaults q_noise=0.01, r_noise=0.5, z_threshold=2.0, and runs backtest.NewKalmanStrategy with a 20-bar lookback.
Data-quality gate	Before simulation, the backend calls ValidateData and rejects a backtest when invalid bars exceed the configured quality threshold.

Software Architecture Diagram



UML DIAGRAMS

→ User Flow Diagram



1. **Visitor arrives** → browses the four public marketing pages (Home, Performance, Pricing, Mission), with Performance having its own live backtest call
2. **Login modal** → User enters existing credentials. System sends POST /auth/login and receives a JWT.
 - **Branch A:** If is_onboarded = true, skip directly to the **Dashboard**.
 - **Branch B:** If is_onboarded = false, route to the **Onboarding Overlay**.
3. **Onboarding check** → new users pick a risk profile and save it, returning users skip straight through
4. **Overview dashboard** → launch/terminate agent actions sit directly off this node
5. **Four app sections** (Overview, Agent Settings, Portfolio, Activity) shown as peers on the layout.
6. **Core Agent Controls (Main View):**
 - **Launch Agent:** Starts the trading agent via POST /api/v1/agent/portfolio/start (submitting strategy_key, risk_profile, symbols, and timeframe).
 - **Terminate Agent:** Stops the running agent via POST /api/v1/agent/portfolio/stop.
 - **Regime Label:** Displays the current market regime directly from the agent's live runtime state.
7. **Agent Settings (/app/agent-settings):**
 - User interacts with risk profile cards and fine-tunes configurations using advanced sliders.
 - System manages configurations via GET/POST /api/v1/user/settings.
8. **Portfolio (/app/portfolio):**
 - User views their asset composition via a donut chart, checks active listings in a positions table, and views their top-performing asset.
 - System fetches data via:
 - a. GET /api/v1/user/portfolio/summary
 - b. GET /api/v1/user/portfolio/positions
 - c. GET /api/v1/user/portfolio/history
9. **Activity (/app/activity):**
 - User reviews live executions in a stream table and checks system alerts in a dedicated sidebar.
 - System fetches data via:
 - a. GET /api/v1/user/portfolio/trades
 - b. GET /api/v1/user/portfolio/alerts

10. **Sign out** → clears token and returns to home

Database Diagram

The schema is designed around three principles drawn from standard relational modeling: referential integrity enforced at the database level rather than the application level, tenant isolation baked into every trading table, and append-only audit trails for anything financially sensitive.

Referential integrity is enforced through foreign keys rather than application-layer checks: a fill cannot exist without a parent order, an order cannot exist without an account, and an account cannot exist without a user. This means the database itself rejects orphaned records, removing an entire class of consistency bugs that would otherwise require defensive application code.

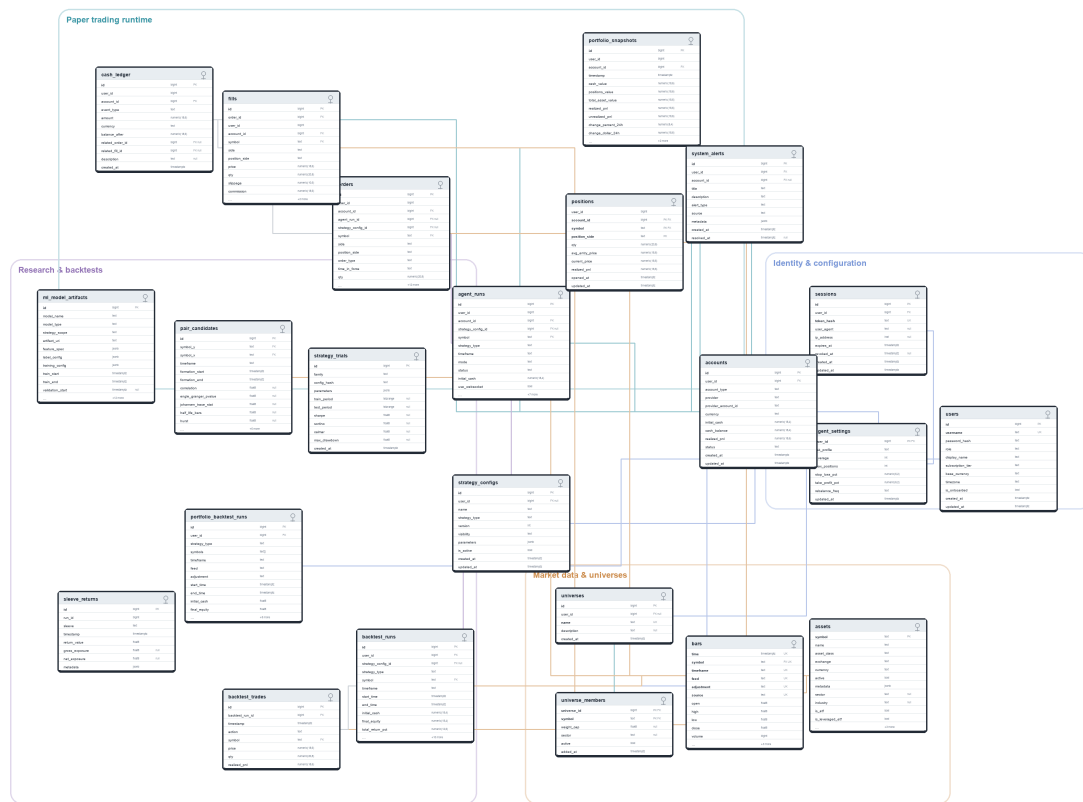
Tenant isolation is enforced through the composite relationship (id, user_id) on the accounts table, which propagates through orders, fills, positions, and snapshots. Every query scopes results to the authenticated user's account, so a misconfigured API call cannot leak one user's trading state to another.

The cash_ledger is append-only by design: cash movements are never updated in place, only inserted as signed entries with a balance_after. This means the full history of every deposit, withdrawal, buy, sell, and commission is preserved and auditable, and the current balance can always be reconstructed by replaying the ledger.

The current migrated schema contains 23 logical application tables, described below using standard ER notation. Relationships are shown as: one-to-many, one-to-one, and optional participation.

O(Alpha) Database Schema

Current migrated PostgreSQL schema: 23 logical tables; bars is range-partitioned separately.



users

The table stores the basic details about each user. A row is entered when a new user registers, and is_onboarded flips to true only after the user has saved matching risk settings and accepted a catalog backtest. The role field distinguishes regular users from admins, subscription_tier records the user plan, and base_currency/timezone personalize display. password_hash stores only the hashed credential, and usernames remain unique on a case-insensitive basis through a lower(username) unique index.

users			
id	bigint	PK	
username	text	UK	
password_hash	text		
role	text		
display_name	text		
subscription_tier	text		
base_currency	text		
timezone	text		
is_onboarded	bool		
created_at	timestampz		
updated_at	timestampz		

sessions

The table stores authentication sessions. token_hash holds the hashed session token so raw tokens are never persisted. expires_at controls natural expiry, revoked_at records explicit invalidation, and user_agent/ip_address retain client audit context. A session is active only while revoked_at is null and expires_at is still in the future.

sessions			♀
id	bigint	PK	
user_id	bigint	FK	
token_hash	text	UK	
user_agent	text	null	
ip_address	inet	null	
expires_at	timestampz		
revoked_at	timestampz	null	
created_at	timestampz		
updated_at	timestampz		

agent_settings

The table stores exactly one saved configuration row per user. It records risk_profile, leverage, max_positions, stop_loss_pct, take_profit_pct, and rebalance_freq. The current onboarding and settings flows validate strategy_key and backtest_accepted in the API request, but those two values are not columns in agent_settings; the selected portfolio catalog key is stored with active runs inside agent_runs.parameters.

agent_settings			♀
user_id	bigint	PK FK	
risk_profile	text		
leverage	int		
max_positions	int		
stop_loss_pct	numeric(6,2)		
take_profit_pct	numeric(6,2)		
rebalance_freq	text		
updated_at	timestampz		

accounts

The table stores trading accounts owned by a user. The default application path uses an internal paper account seeded with `initial_cash`. `cash_balance` tracks available cash and `realized_pnl` accumulates closed-trade P&L. `account_type` separates paper and live-capable account records, `status` controls whether the account can trade, and the composite `(id, user_id)` relationship lets downstream trading tables enforce tenant isolation.

accounts			♀
id	bigint	PK	
user_id	bigint	FK	
account_type	text		
provider	text		
provider_account_id	text		
currency	text		
initial_cash	numeric(18,4)		
cash_balance	numeric(18,4)		
realized_pnl	numeric(18,6)		
status	text		
created_at	timestampz		
updated_at	timestampz		

assets

The table is the instrument catalogue and uses symbol as the primary key. In addition to the original name, asset_class, exchange, currency, active flag, and metadata JSON, the current schema tracks sector, industry, and ETF classification flags: is_etf, is_leveraged_etf, and is_inverse_etf. These fields support universe construction, portfolio screens, and research filters without deleting historical data for inactive assets.

assets			
symbol	text	PK	
name	text		
asset_class	text		
exchange	text		
currency	text		
active	bool		
metadata	jsonb		
sector	text	null	
industry	text	null	
is_etf	bool		
is_leveraged_etf	bool		
is_inverse_etf	bool		
created_at	timestampz		
updated_at	timestampz		

bars

The table stores OHLCV candles and is now keyed by dataset identity rather than only (time, symbol, timeframe). Each row includes feed, adjustment, and source so the system can distinguish raw Alpaca data from adjusted Yahoo/research datasets. A unique dataset index covers (time, symbol, timeframe, feed, adjustment, source), and the table remains range-partitioned by time through bars_y2024, bars_y2025, bars_y2026, and bars_default.

bars			♀
time	timestampz	UK	
symbol	text	FK UK	
timeframe	text	UK	
feed	text	UK	
adjustment	text	UK	
source	text	UK	
open	float8		
high	float8		
low	float8		
close	float8		
volume	bigint		
trade_count	bigint	null	
vwap	float8	null	
created_at	timestampz		

universes

The table stores named asset universes. A universe may be user-scoped through user_id or system-scoped when user_id is null. The name and optional description make curated sets such as the Yahoo100-style default universe addressable by the research and portfolio-agent layers.

universes			♀
id	bigint	PK	
user_id	bigint	FK null	
name	text	UK	
description	text	null	
created_at	timestampz		

universe_members

The table stores the symbols that belong to each universe. The composite primary key (universe_id, symbol) prevents duplicate membership, while weight_cap, sector, active, and added_at provide portfolio-construction metadata and allow a universe to be curated over time without losing its membership history.

universe_members			♀
universe_id	bigint	PK FK	
symbol	text	PK FK	
weight_cap	float8	null	
sector	text	null	
active	bool		
added_at	timestampz		

strategy_configs

The table stores named and versioned strategy configurations. It supports the expanded non-catalog strategy enum from the alpha-foundation migration: MA_CROSSOVER, KALMAN, REGIME_DETECTOR, PAIRS, HMM_ENSEMBLE, ML_META_LABEL, XSEC_MOMENTUM, KALMAN_COINTEGRATION, and MULTI_ENGINE_ENSEMBLE. PORTFOLIO_CATALOG is intentionally not part of this table constraint; catalog strategy selection is represented by strategy_key inside agent_runs.parameters.

strategy_configs			♀
id	bigint	PK	
user_id	bigint	FK null	
name	text		
strategy_type	text		
version	int		
visibility	text		
parameters	jsonb		
is_active	bool		
created_at	timestampz		
updated_at	timestampz		

backtest_runs

The table provides schema support for persisted single-symbol or legacy backtest histories. It records the run window, symbol, timeframe, initial and final equity, headline metrics, parameters, diagnostics, and the equity curve JSON. The current portfolio streaming/onboarding backtest path returns results live from /api/v1/backtest/stream and does not persist PORTFOLIO_CATALOG runs into this constrained table.

backtest_runs			
id	bigint	PK	
user_id	bigint	FK	
strategy_config_id	bigint	FK null	
strategy_type	text		
symbol	text	FK	
timeframe	text		
start_time	timestampz		
end_time	timestampz		
initial_cash	numeric(18,4)		
final_equity	numeric(18,4)		
total_return_pct	numeric(12,6)		
annual_return_pct	numeric(12,6)		
sharpe_ratio	numeric(12,6)		
sortino_ratio	numeric(12,6)		
max_drawdown_pct	numeric(12,6)		
num_trades	int		
parameters	jsonb		
equity_curve	jsonb		
diagnostics	jsonb		
status	text		
created_at	timestampz		

backtest_trades

The table provides schema support for trade-level backtest history linked to backtest_runs. It records timestamp, action, symbol, price, quantity, and realized P&L for simulated trades. In the current codebase, portfolio backtest trades are returned in the portfolio result payload; this table should be described as persistence capacity rather than the live source of onboarding chart data unless the save path is reconnected.

backtest_trades			♀
id	bigint	PK	
backtest_run_id	bigint	FK	
timestamp	timestamptz		
action	text		
symbol	text	FK	
price	numeric(18,6)		
qty	numeric(20,8)		
realized_pnl	numeric(18,6)		

portfolio_backtest_runs

The table is the newer schema for multi-symbol portfolio backtest results. It stores symbols, timeframe, feed, adjustment, run window, initial/final equity, metrics, parameters, equity_curve, and trades as JSON. It is the correct destination for persisted PORTFOLIO_CATALOG/XSEC/Kalman-pair portfolio histories when persistence is wired, while today the onboarding graph primarily consumes streamed NDJSON results.

portfolio_backtest_runs			♀
id	bigint	PK	
user_id	bigint	FK	
strategy_type	text		
symbols	text[]		
timeframe	text		
feed	text		
adjustment	text		
start_time	timestamptz		
end_time	timestamptz		
initial_cash	float8		
final_equity	float8		
metrics	jsonb		
parameters	jsonb		
equity_curve	jsonb		
trades	jsonb		
created_at	timestamptz		

sleeve_returns

The table stores per-sleeve return and exposure series for portfolio research. run_id identifies the logical run, sleeve names the component strategy or allocation sleeve, timestamp orders the series, and gross_exposure/net_exposure provide diagnostics for ensemble or active-sleeve analysis. metadata keeps additional research annotations as JSON.

sleeve_returns			♀
id	bigint	PK	
run_id	bigint		
sleeve	text		
timestamp	timestampz		
return_value	float8		
gross_exposure	float8	null	
net_exposure	float8	null	
metadata	jsonb		

ml_model_artifacts

The table is a registry for model metadata, not a blob store. It records model_name, model_type, strategy_scope, artifact_uri, feature/label/training specs, train and validation windows, validation metrics such as AUC/logloss/Sharpe/DSR/PBO, leaves-parity checks, and status. The promoted artifact bytes live in local/object storage and are loaded by the API from the configured artifact root.

ml_model_artifacts			
id	bigint	PK	
model_name	text		
model_type	text		
strategy_scope	text		
artifact_uri	text		
feature_spec	jsonb		
label_config	jsonb		
training_config	jsonb		
train_start	timestampz		
train_end	timestampz		
validation_start	timestampz	null	
validation_end	timestampz	null	
auc	float8	null	
logloss	float8	null	
sharpe_net	float8	null	
sortino_net	float8	null	
max_drawdown_pct	float8	null	
dsr	float8	null	
pbo	float8	null	
leaves_parity_max_abs_error	float8	null	
leaves_parity_passed	bool		
status	text		
created_at	timestampz		

pair_candidates

The table stores Kalman/cointegration pair research candidates. symbol_y and symbol_x reference assets, formation_start and formation_end define the research window, and statistical diagnostics such as correlation, Engle-Granger p-value, Johansen trace statistic, half-life, Hurst, and estimated trading cost support offline approval. approved/status gate whether a pair can graduate from research.

pair_candidates			🔗
id	bigint	PK	
symbol_y	text	FK	
symbol_x	text	FK	
timeframe	text		
formation_start	timestampz		
formation_end	timestampz		
correlation	float8	null	
engle_granger_pvalue	float8	null	
johansen_trace_stat	float8	null	
half_lifeBars	float8	null	
hurst	float8	null	
avg_spread_bps	float8	null	
estimated_round_trip_cost_bps	float8	null	
approved	bool		
status	text		
metadata	jsonb		
created_at	timestampz		

strategy_trials

The table stores research trial metadata for alpha-search experiments. family and config_hash identify the candidate family/configuration, parameters captures the tested settings, train_period and test_period preserve the walk-forward window, and sharpe/sortino/calmar/max_drawdown store summary validation metrics for comparison across trials.

strategy_trials			🔗
id	bigint	PK	
family	text		
config_hash	text		
parameters	jsonb		
train_period	tstzrange	null	
test_period	tstzrange	null	
sharpe	float8	null	
sortino	float8	null	
calmar	float8	null	
max_drawdown	float8	null	
created_at	timestampz		

agent_runs

The table represents a paper or live-capable automated agent session. The portfolio-agent path creates PORTFOLIO_CATALOG rows, uses symbol as the benchmark symbol, and stores the concrete strategy_key, selected symbols, paper_only/deployment metadata, execution_mode, and runtime_state inside parameters. status moves through starting, running, stopping, stopped, or failed, while last_heartbeat_at, stopped_at, and stop_reason make running-state and restart reconciliation auditable.

agent_runs			♀
id	bigint	PK	
user_id	bigint		
account_id	bigint	FK	
strategy_config_id	bigint	FK null	
symbol	text	FK	
strategy_type	text		
timeframe	text		
mode	text		
status	text		
initial_cash	numeric(18,4)		
use_websocket	bool		
parameters	jsonb		
started_at	timestampz		
stopped_at	timestampz	null	
last_heartbeat_at	timestampz	null	
stop_reason	text	null	
created_at	timestampz		
updated_at	timestampz		

orders

The table stores every submitted paper/broker order. It captures side, position_side, order_type, time_in_force, quantity, optional limit/stop prices, provider identifiers, metadata, and lifecycle timestamps. client_order_id is unique when present and is used by the DB execution router as an idempotency key so repeated portfolio-agent evaluations do not double-fill the same intended trade.

orders			
id	bigint	PK	
user_id	bigint		
account_id	bigint	FK	
agent_run_id	bigint	FK null	
strategy_config_id	bigint	FK null	
symbol	text	FK	
side	text		
position_side	text		
order_type	text		
time_in_force	text		
qty	numeric(20,8)		
limit_price	numeric(18,6)	null	
stop_price	numeric(18,6)	null	
status	text		
client_order_id	text	UK null	
provider_order_id	text	null	
submitted_at	timestampz		
filled_at	timestampz	null	
canceled_at	timestampz	null	
rejected_reason	text	null	
metadata	jsonb		
created_at	timestampz		
updated_at	timestampz		

fills

The table stores executions against orders. Each fill links to its parent order and account, records symbol, side, position_side, executed price, quantity, slippage, commission, liquidity, and filled_at. The current DB execution router writes filled paper orders and fills together so the activity screen can read a canonical execution stream.

fills			♀
id	bigint	PK	
order_id	bigint	FK	
user_id	bigint		
account_id	bigint	FK	
symbol	text	FK	
side	text		
position_side	text		
price	numeric(18,6)		
qty	numeric(20,8)		
slippage	numeric(10,6)		
commission	numeric(18,6)		
liquidity	text		
filled_at	timestampz		

positions

The table stores the current open holdings of an account and is keyed on (account_id, symbol, position_side), allowing at most one long and one short row per symbol/account. It tracks quantity, average entry, current marked price, realized P&L, and timestamps. The portfolio dashboard allocation reads this table through /api/v1/user/portfolio/positions.

positions			♀
user_id	bigint		
account_id	bigint	PK FK	
symbol	text	PK FK	
position_side	text	PK	
qty	numeric(20,8)		
avg_entry_price	numeric(18,6)		
current_price	numeric(18,6)		
realized_pnl	numeric(18,6)		
opened_at	timestampz		
updated_at	timestampz		

cash_ledger

The table is an append-only cash audit trail. It records deposits, withdrawals, buy/sell trade cash movements, commissions, and adjustments with signed amount and balance_after. Related order/fill links make each cash movement traceable back to the execution event that caused it.

cash_ledger			♀
id	bigint	PK	
user_id	bigint		
account_id	bigint	FK	
event_type	text		
amount	numeric(18,6)		
currency	text		
balance_after	numeric(18,6)		
related_order_id	bigint	FK null	
related_fill_id	bigint	FK null	
description	text	null	
created_at	timestampz		

portfolio_snapshots

The table stores account valuation snapshots written by the portfolio repository/worker after evaluations and mark-to-market updates. It captures cash value, positions value, total asset value, realized/unrealized P&L, 24-hour change values, and target progress. estimated_annual_yield exists in the schema but is currently written as 0 by SavePortfolioSnapshot, so it should not be marketed as a live computed metric yet.

portfolio_snapshots			♀
id	bigint	PK	
user_id	bigint		
account_id	bigint	FK	
timestamp	timestampz		
cash_value	numeric(18,6)		
positions_value	numeric(18,6)		
total_asset_value	numeric(18,6)		
realized_pnl	numeric(18,6)		
unrealized_pnl	numeric(18,6)		
change_percent_24h	numeric(8,4)		
change_dollar_24h	numeric(18,6)		
estimated_annual_yield	numeric(18,6)		
target_progress_percent	numeric(8,4)		

system_alerts

The table stores user-visible notifications. Portfolio-agent actions write alerts for rebalance executions and user-requested stops, with source and metadata identifying where the alert came from. alert_type is constrained to INFO, WARNING, or CRITICAL, and resolved_at distinguishes open alerts from cleared ones.

system_alerts			♀
id	bigint	PK	
user_id	bigint	FK	
account_id	bigint	FK null	
title	text		
description	text		
alert_type	text		
source	text		
metadata	jsonb		
created_at	timestampz		
resolved_at	timestampz	null	

Technical Overview

Kalman filter and State-space model

The engine extracts a smoothed “true price” from a noisy close-price feed by running a discrete, scalar, linear state-space filter. Let $\hat{x}_t$ be the estimated latent true price at bar t and let P_t be the scalar estimation error covariance. The implementation assumes a random-walk price process with no control input, unit state transition, and unit observation:

$$x_t = x_{t-1} + w_t, \quad z_t = x_t + v_t,$$

with process noise $w_t \sim \mathcal{N}(0, Q)$ and measurement noise $v_t \sim \mathcal{N}(0, R)$.

The two variances are the only tunable parameters:

- Q (the process-noise variance, exposed as `q_noise`) governs how quickly the filter adapts to genuine price movement.

- R (the measurement-noise variance, exposed as `r_noise`) governs how aggressively short-term market noise is rejected.

Both are configurable per run from the agent settings panel.

Prediction (time update)

With unit state transition and no control input, the projection of state and the extrapolation of error covariance reduce to:

$$\hat{x}_{t|t-1} = \hat{x}_{t-1|t-1}$$

$$P_{t|t-1} = P_{t-1|t-1} + Q$$

Correction (measurement update)

On receipt of the raw close z_t , the filter computes the Kalman gain K_t , blends the prediction with the measurement, and contracts the error covariance:

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t (z_t - \hat{x}_{t|t-1})$$

$$P_{t|t} = (1 - K_t) P_{t|t-1}$$

The gain $K_t \in (0,1)$ interpolates between trusting the prior ($K_t \rightarrow 0$ when R dominates) and trusting the latest print ($K_t \rightarrow 1$ when $P_{t|t-1}$ dominates).

Initialization and steady state

The filter is seeded on its first observation rather than from a prior guess:

$$\hat{x}_{0|0} = z_0, \quad P_{0|0} = 1.$$

Because Q and R are constant, P_t converges to a fixed point and K_t approaches a constant steady-state gain; the filter then behaves like an exponential moving average whose smoothing factor is set by the ratio Q/R .
Default configuration: $Q = 0.01$, $R = 0.5$.

Kalman mean-reversion signal (Z-score)

Residual

Execution signals are derived from the statistical residual between the observed price and the filter estimate. The deployed code evaluates the residual against the **posterior** (filtered) estimate $\hat{x}_{t|t}$, that is, it calls the filter update first and then measures the gap:

$$e_t = z_t - \hat{x}_{t|t}.$$

Implementation note. This is the filtered residual, not the classical one-step innovation $z_t - \hat{x}_{t|t-1}$. The distinction matters for tuning: because the posterior has already absorbed part of z_t , the residual magnitude scales with $(1 - K_t)$ relative to the innovation. If a future revision wants the canonical innovation, capture the residual before the update step.

Rolling dispersion

The residuals are accumulated in a sliding window of the most recent W observations (*lookbackWindow*, default $W = 20$). Over that window the engine computes the sample standard deviation, using Bessel's correction:

$$\sigma_W = \sqrt{\frac{1}{W-1} \sum_{i=t-W+1}^t (e_i - \bar{e})^2}, \quad \bar{e} = \frac{1}{W} \sum_{i=t-W+1}^t e_i.$$

The window mean $\bar{e}$ is used only to estimate dispersion; the score itself (below) divides the raw residual by σ_W , since the residual stream is expected to be approximately zero-mean.

Z-score and decision rule

The standardized deviation that triggers entries and exits is:

$$Z_t = \frac{e_t}{\sigma_W}.$$

Signals are assigned deterministically against a symmetric threshold Z_{thr} (`z_threshold`, default 2.0):

$$\text{Signal}_t = \begin{cases} \text{BUY}, & Z_t < -Z_{\text{thr}} \\ \text{SELL}, & Z_t > +Z_{\text{thr}} \\ \text{HOLD}, & \text{otherwise.} \end{cases}$$

The logic is mean-reverting: a price far **below** the filtered estimate ($Z_t < -Z_{\text{thr}}$) is treated as oversold and triggers a BUY, while a price far **above** it triggers a SELL.

Warm-up and numerical guards

Two guards precede any trade. While fewer than W residuals have accumulated, the strategy emits HOLD (warm-up). If the window standard deviation evaluates to $\sigma_W = 0$ (a degenerate, perfectly flat window), the division is skipped and the bar is forced to HOLD to avoid division by zero.

Moving-average crossover

Simple moving average

For a window of n bars, the simple moving average of the closing price is:

$$\text{SMA}_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} z_{t-i}.$$

The implementation computes this incrementally (a rolling sum that adds the incoming close and subtracts the one leaving the window), so each update is $O(1)$. Indices before $t = n - 1$ are undefined and stored as zero; they are never used for signals (see §4.3).

Crossover conditions

The model tracks a fast and a slow average with periods $n_f < n_s$ (defaults $n_f = 10$, $n_s = 30$).

A signal fires only on the bar where the two lines actually cross, detected by comparing the current and previous relationship:

$$\begin{aligned} c_t^\uparrow: & \text{SMA}_{n_f}(t-1) \leq \text{SMA}_{n_s}(t-1) \wedge \text{SMA}_{n_f}(t) > \text{SMA}_{n_s}(t) \quad (\text{golden cross}) \\ c_t^\downarrow: & \text{SMA}_{n_f}(t-1) \geq \text{SMA}_{n_s}(t-1) \wedge \text{SMA}_{n_f}(t) < \text{SMA}_{n_s}(t) \quad (\text{death cross}) \end{aligned}$$

Decision rule and position state

The strategy is long-only and carries a single position flag. The decision rule is:

$$\text{Signal}_t = \begin{cases} \text{BUY}, & c_t^\uparrow \\ \text{SELL}, & c_t^\downarrow \\ \text{HOLD}, & \text{otherwise.} \end{cases}$$

A golden cross opens (or maintains) the long and sets the position flag to long; a death cross flattens it. Between crosses, while long, the bar is explicitly held. The detection loop begins at $t = n_s$ rather than $t = n_s - 1$, which guarantees that both the current and the previous SMA pair are drawn from fully populated windows and never from the zero-padded warm-up region.

Parameters and validity

The strategy rejects degenerate configurations: it requires $n_f > 0$, $n_s > 0$, and strictly $n_f < n_s$, and it needs at least $n_s + 1$ bars of history before it produces signals.

Execution semantics

Back test fill model

The back test engine is deliberately conservative about look-ahead. A signal computed from data up to and including bar t is executed at the **open of bar** $t + 1$:

$$\text{fill price}_t = \text{Open}_{t+1}.$$

Position management is all-in / all-out and long-only. On a BUY from flat, the full cash balance is converted to shares; on a SELL from long, the entire holding is liquidated:

$$\text{shares} \leftarrow \frac{\text{cash}}{\text{Open}_{t+1}} \quad (\text{BUY}), \quad \text{cash} \leftarrow \text{shares} \cdot \text{Open}_{t+1} \quad (\text{SELL}).$$

Portfolio paper-agent sizing

The current portfolio paper-trading path uses target-weight reconciliation rather than fixed 10% buys or half-position sells. The DB-backed execution router reads current cash and positions, computes desired long quantities from portfolio target weights, sells reductions before buys, skips shorts in v1, and records every paper fill with a deterministic client_order_id so repeated evaluations do not double-fill.

$$q_{\text{buy}} = \frac{0.10 \cdot \text{cash}}{z_t}, \quad q_{\text{sell}} = \max(1, 0.50 \cdot \text{position}).$$

Performance and risk metrics

Every backtest returns a standard risk/return summary computed from the equity curve $\{E_0, E_1, \dots, E_T\}$ Per-bar simple returns are:

$$r_i = \frac{E_i - E_{i-1}}{E_{i-1}}, \quad i = 1, \dots, T.$$

Total return is measured end-to-end:

$$R_{\text{total}} = \frac{E_T - E_0}{E_0}.$$

Annualized Sharpe ratio (zero risk-free rate, sample standard deviation of returns, 252 trading days):

$$\text{Sharpe} = \frac{\bar{r}}{\sigma_r} \sqrt{252}, \quad \sigma_r = \sqrt{\frac{1}{T-1} \sum_{i=1}^T (r_i - \bar{r})^2}.$$

Annualized Sortino ratio replaces total volatility with downside deviation, penalizing only negative returns (n_- is the count of negative returns, target return 0):

$$\text{Sortino} = \frac{\bar{r}}{\sigma_d} \sqrt{252}, \quad \sigma_d = \sqrt{\frac{1}{n_-} \sum_{r_i < 0} r_i^2}.$$

Maximum drawdown is the largest peak-to-trough decline of the equity curve:

$$\text{MaxDD} = \max_{0 \leq t \leq T} \left(\frac{\max_{0 \leq s \leq t} E_s - E_t}{\max_{0 \leq s \leq t} E_s} \right).$$

Sharpe and Sortino return 0 when their denominators are zero (insufficient variation), guarding against division by zero on flat or all-positive curves.

Strategy Catalogs

Public backend routes expose multiple strategy families:

POST /api/v1/backtest supports MA_CROSSOVER, KALMAN, ML_META_LABEL, PORTFOLIO_CATALOG, XSEC_MOMENTUM, and KALMAN_COINTEGRATION;

POST /api/v1/backtest/stream supports portfolio strategies; POST

/api/v1/agent/start supports MA_CROSSOVER, KALMAN, HMM/HMM_ENSEMBLE, and ML_META_LABEL.

$$\text{TR}_j = \max(H_j - L_j, |H_j - z_{j-1}|, |L_j - z_{j-1}|), \quad v_t = \frac{1}{L} \sum_{j=t-L+1}^t \frac{\text{TR}_j}{z_j}.$$

HMM_ENSEMBLE is an API-exposed agent mode, not merely dormant code. It combines MA and Kalman votes, then uses an HMM risk overlay for regime state, confidence, and exposure/risk context. The newer portfolio worker separately computes an HMM overlay state on the benchmark symbol and writes it into agent_runs.parameters.runtime_state for the dashboard regime tracker.

$$\text{Regime}_t = \begin{cases} \text{VOLATILE}, & v_t > v_{\text{thr}} \\ \text{BULL}, & v_t \leq v_{\text{thr}} \wedge \text{SMA}_{n_f}(t) > \text{SMA}_{n_s}(t) \\ \text{BEAR}, & v_t \leq v_{\text{thr}} \wedge \text{SMA}_{n_f}(t) < \text{SMA}_{n_s}(t) \\ \text{NEUTRAL}, & \text{otherwise.} \end{cases}$$

ML_META_LABEL is also API-exposed. It wraps a MA/Kalman base strategy, loads artifacts from model_path, metadata_path, or model_registry_root, builds point-in-time features, applies calibration and thresholds, and can veto or size BUY signals before they reach the backtest or paper-agent worker.

The multi-symbol portfolio research surface is separate from the single-symbol agent surface.

- PORTFOLIO_CATALOG constructs a named catalog strategy by strategy_key.
- XSEC_MOMENTUM constructs a cross-sectional momentum portfolio.
- KALMAN_COINTEGRATION constructs a pair-trading strategy with distinct symbol_y and symbol_x parameters.

Parameter reference

Symbol	Meaning	Field	Default
Q	Process-noise variance	<i>q_noise</i>	0.01
R	Measurement-noise variance	<i>r_noise</i>	0.5
W	Residual lookback window	<i>lookbackWindow</i>	20
Z_{thr}	Z-score entry/exit threshold	<i>z_threshold</i>	2.0
n_f	Fast SMA period	<i>fast_period</i>	10
n_s	Slow SMA period	<i>slow_period</i>	30
$P_{0 0}$	Initial error covariance	(constant)	1.0
—	Annualization factor	(constant)	$\sqrt{252}$

All figures and equations reflect the implementation as of this revision. The Sharpe, Sortino, and drawdown values produced by the engine are model outputs on historical data and are illustrative, not investment advice.

Portfolio catalog strategy path

The main paper-trading path is implemented as PORTFOLIO_CATALOG, a multi-symbol PortfolioStrategy path. Catalog specs are returned by GET `/api/v1/strategies/catalog` and are constructible through `NewStrategyFromCatalog` for `/api/v1/backtest`, `/api/v1/backtest/stream`, and `/api/v1/agent/portfolio/start`. The API returns a curated Yahoo100-style equity/ETF universe anchored on VOO. Each StrategySpec includes key, display_name, family, risk_profile, deployment_status, promoted_checkpoint, requires_model_artifacts, paper_only, benchmark_symbol, evidence_paths, and notes. All current catalog entries are paper-only.

Current catalog entries

Key	Risk	Family / implementation	Status and documentation note
lgbm_ranker_h63_low	low	benchmark_lgbm_ranker_h63; learned h63 sleeve, 5% active, top 1	conservative_variant; requires artifacts; paper-only conservative sizing
ranker_proxy_h63_low	low	benchmark_ranker_proxy_h63; deterministic h63 proxy, 8% sleeve, top 2	conservative_variant; no model artifacts; paper-only low-risk proxy
lowvol_sleeve_low	low	benchmark_lowvol; low-volatility stock sleeve	rejected_diagnostic; defensive comparison only
lgbm_ranker_h63_medium	medium	benchmark_lgbm_ranker_h63; learned h63 sleeve, 15% active, top 3	promoted_research_checkpoint; requires artifacts; strongest current checkpoint but still paper-only
ranker_proxy_h63_medium	medium	benchmark_ranker_proxy_h63; deterministic h63 proxy, 15% sleeve, top 3	promoted_research_checkpoint; paper-only; lower confidence than learned h63
ranked_sleeve_medium	medium	benchmark_ranked_sleeve; broader risk-budgeted sleeve	rejected_diagnostic; fallback/diagnostic comparison
lgbm_ranker_h63_high	high	benchmark_lgbm_ranker_h63; learned h63 sleeve, 25% active, top 5	experimental_variant; requires artifacts; paper comparison only
benchmark_tsmom_high	high	benchmark_tsmom; ETF and broad- universe time-series momentum sleeves	rejected_diagnostic; high-risk challenger only
composite_momentum_high	high	composite_momentum; ETF 21-bar plus stock 126-bar momentum legs	rejected_diagnostic; high-active-weight comparison only

DB-backed paper execution router

DBExecutionRouter converts portfolio targets into persisted paper-trading state.

On every evaluation it reads account cash and open long positions, computes desired quantities from target weights, sells reductions before buys, records BUY_LONG and SELL_LONG fills through RecordLongFillKeyed, updates cash and positions transactionally, marks latest prices, writes portfolio_snapshots, and emits system_alerts when rebalances or risk exits occur.

Idempotency is enforced through deterministic client_order_id values such as runID:symbol:barTime:BUY and runID:symbol:barTime:SELL. If the same evaluation is retried, the repository detects the existing client_order_id and avoids double-filling the account.

Portfolio worker run loop

The portfolio orchestrator enforces one active portfolio agent per user. On launch, it creates a PORTFOLIO_CATALOG agent_run, warms up aligned 1Day bars through GetBarsMulti, attempts an Alpaca refresh when credentials exist, evaluates immediately, then polls on the configured cadence. Each successful loop updates runtime state and last_heartbeat_at.

On API startup, the orchestrator attempts to resume active portfolio catalog runs. Stale non-portfolio agent runs are reconciled as failed with orphaned_on_restart. This distinction is important when documenting restart behavior.

HMM regime runtime overlay

The dashboard regime label is now wired to the portfolio worker runtime state. The worker computes an HMM risk-overlay state on the benchmark symbol and stores regime_label, confidence, probability_low, probability_medium, probability_high, overlay_role, overlay_multiplier, overlay_confidence, model_healthy, and bar_time inside agent_runs.parameters.runtime_state. The dashboard reads this through /api/v1/agent/list.

Streaming portfolio backtests

The onboarding and Agent Settings backtest graph uses POST /api/v1/backtest/stream with PORTFOLIO_CATALOG payloads. The endpoint returns application/x-ndjson events: started, progress, completed, and error. Streaming is restricted to portfolio strategies (PORTFOLIO_CATALOG, XSEC_MOMENTUM, KALMAN_COINTEGRATION); single-symbol MA/Kalman/ML backtests use POST /api/v1/backtest.

Architectural philosophy

O(Alpha) is built around a small number of deliberate engineering principles: a strictly layered architecture with clear separation of concerns, a disciplined application of DRY and KISS, concurrency that isolates failure, and data integrity enforced by construction. The goal is a codebase that is cohesive, predictable, and cheap to extend — where a new strategy, endpoint, or screen slots into an existing seam rather than threading logic through unrelated modules.

Decoupled architecture and separation of concerns

The platform separates presentation, request orchestration, domain logic, and data persistence into independent layers. Each layer depends only on the contract of the layer beneath it, never on its internals, so a change in one tier (say, swapping a SQL query or re-skinning a screen) does not ripple outward.

System dataflow path

1. **Presentation.** The Next.js client (*frontend/src/app*) is the sole user gateway.
2. **Transport.** The client talks to the backend exclusively over asynchronous REST calls using HTTP and JSON.
3. **Orchestration.** A Go service built on the Gin web framework (*backend/internal/api*) routes and validates every request.
4. **Connection isolation.** Database access is funneled through a *jackc/pgx/v5* connection pool (*pgxpool*), so handlers never manage raw connections.
5. **Persistence.** State is stored in a normalized, constraint-enforced PostgreSQL schema hosted on Supabase.

Core architecture components

Layer	Location	Responsibility
Presentation (View)	<i>frontend/src/app</i>	Renders the UI, hydrates client state through declarative data hooks (SWR), and dispatches user actions as REST requests.
Orchestration (Controller)	<i>backend/internal/api</i>	Routes HTTP traffic through Gin, binds and validates JSON payloads, applies auth middleware, and maps endpoints to handlers.
Domain Logic (Service)	<i>backend/internal/agent</i> , <i>backend/internal/backtest</i>	Runs strategy computations and noise filtering (e.g. the Kalman filter), simulates backtests, and manages live paper-trading sessions.
Persistence (Model)	<i>backend/internal/db</i> , <i>backend/pkg/models</i>	Owns entity definitions, manages the <i>pgxpool</i> connection pool, and executes batched, transactional operations against Postgres.

Architectural segregation

- **Persistence and data modeling.** All SQL lives behind an isolated data-access layer (*backend/internal/db*), while *backend/pkg/models* defines the typed entities (*Bar*, *User*, *BacktestRequest*, *BacktestResult*, and so on). Go's static typing guarantees shape correctness in the application, and native database constraints — composite primary keys (e.g. *bars* keyed on *time*, *symbol*, *timeframe*), unique constraints, and foreign keys — guarantee referential integrity at the storage level. Schema decisions are governed by the Supabase Postgres best-practices ruleset vendored into the repo under *backend/agents/skills*.
- **Presentation and interface isolation.** The client is fully decoupled from data mutations. The UI never reaches the database directly; it changes application state only by dispatching structured REST requests to designated backend routes, which keeps the rendering layer ignorant of persistence details.
- **API orchestration sub-layers.** Within the backend, responsibilities are split across three clear roles: **Routers** (*router.go*) declare entry points and cross-origin permissions, **Handlers** validate payloads and resolve the authenticated user (e.g. *deriveUserIDQuery*), and **Repositories** perform the low-level data executions. A request flows Router → Handler → Repository and back, with each role testable in isolation.

Design patterns: DRY & KISS

To keep maintenance overhead low, both the client and the backend services are unified around two standard principles.

DRY: Don't Repeat Yourself

Shared behavior is consolidated into reusable, parameterized building blocks rather than copied across modules.

- **UI component standardization.** Data-heavy widgets — real-time performance panels, active-position tables, and risk-adjustment controls — are composed from a shared set of card and layout primitives. A visual change made once to a base component cascades across every screen that uses it.
- **State abstraction hooks.** Reusable functional hooks distribute reactive state and handle cross-cutting concerns such as polling and revalidation, freeing individual screens to focus only on their unique content and interactions.
- **Two strategy contracts for different execution scopes.** The backend separates single-symbol and portfolio-level strategy logic. `backtest.Strategy` refers to the contract for single-instrument simulations such as MA crossover and Kalman strategies, where signals are generated from one symbol's bar stream. For multi-symbol portfolio workflows, the system uses `backtest.PortfolioStrategy`, which powers the portfolio catalog strategies and the main paper-agent execution flow. This split keeps simple backtests lightweight while giving the live agent a contract designed for universe-level allocation, rebalancing, and risk-aware execution.
- **Shared transactional helpers.** Repository routines such as `ensureAssetTx`, `ensureDefaultPaperAccountTx`, `updateAccountCashTx`, and `insertCashLedgerTx` are written once and reused across order fills, agent-run creation, and user provisioning, so the rules for "ensure an account exists" or "write a cash-ledger entry" live in exactly one place.

KISS — Keep It Simple

Backend interaction paths favor legibility and deterministic behavior to reduce runtime surprises.

- **Linear execution routes.** Endpoints avoid deeply nested or circular control flow. A request maps directly to one handler that runs a clear, ordered

sequence. *RunBacktest*, for example, proceeds strictly: bind the request, validate the strategy parameters, fetch bars, gate on data quality, select the strategy, run the simulation, and return the result.

- **Granular responsibility delegation.** Complex routines are decomposed into small, single-purpose functions. The ingest service's delta-sync loop is split so that *syncAllSymbolsDelta* fans work out per symbol to *syncSingleSymbolDelta*, which in turn performs one legible pipeline — read the latest stored timestamp, compute the missing window, fetch from the market-data API, then upsert — making each step independently testable and easy to diagnose.

Concurrency, isolation, and lifecycle safety

Because live agents run as long-lived background work, concurrency is treated as a first-class design concern rather than an afterthought.

- **Isolated workers.** The *AgentManager* keeps a registry of active workers in a *map[string]*AgentWorker* guarded by a *sync.RWMutex*, keyed per user and symbol (*userID_symbol*). This guarantees one worker per user-symbol pair and prevents duplicate agents from racing on the same position.
- **Lifecycle decoupling.** Each *AgentWorker* runs under its own cancellable context that is intentionally detached from the originating HTTP request, communicates through dedicated *done* and *error* channels, and uses *sync.Once* so that stopping is idempotent. Its in-memory bar buffer is bounded and guarded by its own mutex.
- **Safe shared state.** The simulated *PaperAccount* serializes all reads and writes behind a *sync.RWMutex*, so concurrent equity reads and trade writes cannot corrupt cash or position balances.
- **Graceful shutdown.** The API and ingest binaries listen for *SIGINT/SIGTERM* and shut down within a bounded timeout, allowing in-flight work to drain cleanly.

Data integrity by construction

The persistence layer is designed so that correctness is the default outcome, not something callers must remember to enforce.

- **Atomic transactions.** Multi-statement writes — recording a fill while updating cash, positions, and the ledger; creating an agent run; provisioning

a new user and their baseline account — are wrapped in a single transaction with a deferred rollback, so a partial failure leaves no half-written state.

- **Idempotent upserts.** Ingestion and configuration writes use *INSERT ... ON CONFLICT* (for *bars*, *assets*, *agent_settings*, and *positions*), so re-running a sync or re-saving settings converges to the correct state rather than duplicating rows.
- **Quality gates.** Backtests refuse to run on untrustworthy history: *ValidateData* aborts the request if more than 5% of bars in range fail integrity checks.
- **No look-ahead.** The backtest engine executes a signal computed at bar t against the open of bar $t+1$, structurally preventing future information from leaking into past decisions.

Project Management and Version Control

Engineering discipline extends beyond the codebase itself. The repository is managed through a structured Git workflow designed to keep main always deployable and make the contribution history legible.

All feature work is developed on short-lived branches named by scope (e.g. *feat/hmm-regime-overlay*, *fix/portfolio-snapshot-race*). Pull requests are the only path into main: direct pushes are blocked. Each PR must pass the full CI suite (lint, type-check, build, tests) before it is eligible for review, and at least one approval is required before merge. This enforces a two-person awareness on every change to the infrastructure.

GitHub Issues are used to track work at the task level, with labels separating feature, bug, and infrastructure work. Milestones map directly to the Orbital milestone schedule, so the issue backlog reflects planned deliverables rather than an unstructured to-do list. Larger features such as the HMM regime layer and the portfolio execution router are broken into sub-issues linked to a parent, giving visibility into partial progress without waiting for the entire feature to land.

This workflow means the git log is a legible record of what changed and why, PRs serve as lightweight design documents, and rollbacks to any prior milestone state is a single revert operation.

Quality Assurance

Every pull request runs an automated suite of checks via GitHub Actions. The main branch is protected: a PR can only be merged once all required checks pass ("green"), keeping main continuously releasable.

Quality control stages

As this is a polyglot monorepo, checks run per stack. *Frontend (Next.js / TypeScript)*: ESLint enforces shared lint rules, Prettier enforces consistent formatting (applied on save), and tsc performs static type-checking so type errors surface before runtime. *Backend (Go)*: gofmt enforces formatting, go vet/golangci-lint catch suspect constructs, and go build verifies compilation. *Tests*: the automated unit-test suite (go test for the backend strategies, backtest engine, and repositories; the frontend test runner) must pass on every PR.

Continuous integration

On each pull request, GitHub Actions runs the lint, format, type-check, and test stages above across both stacks. A PR cannot merge until all are green, giving a safe path into main.

Continuous deployment

Opening a PR triggers a Vercel preview deployment of the frontend (and a container build of the backend), so deployment is validated before merge. Once the PR is green and merged, production deploys automatically: Vercel promotes the frontend, and the backend containers (api, ingest, migrate) roll out to the production host.

Unit Testing

Terminal Module Boundary	Target Endpoint Route	Protocol Method	Core Technical Test Verification Metric
SECURITY	/auth/login	Mutation (POST)	Confirms a 401 Unauthorized status code is raised if the incoming credential payload fails to match active database signatures.
SECURITY	/auth/login	Mutation (POST)	Verifies a successful login returns a valid JWT signature string alongside the user configuration object.
SECURITY	/auth/validate	Mutation (POST)	Asserts that tampered, modified, or expired token structures are rejected with an explicit validation error token.

SECURITY	/auth/me	Query (GET)	Confirms a valid session token safely extracts and returns the calling user's profile metadata from the database layer.
BACKTEST	/api/v1/backtest REQUEST: <pre> { "symbol": "AAPL", "strategy_type": "MA_CROSSOVER KALMAN", "timeframe": "1Day", "start": "ISO8601", "end": "ISO8601", "fast_period": 10, "slow_period": 30, "q_noise": 0.01, "r_noise": 0.5, "z_threshold": 2.0, "initial_cash": 100000 </pre>	Mutation (POST)	Asserts that passing an invalid asset ticker index returns a 404 Not Found error state. Verifies that a valid simulation request maps historical bar responses through the selected strategy execution filter cleanly.

	<pre> } RESPONSE: { "symbol", "equity_curve", "final_equity", "sharpe", "sortino", "max_drawdown", "total_return", "num_trades" } </pre>		
BACKTEST	/api/v1/backtest	Mutation (POST)	Checks that setting a fast MA period greater than or equal to a slow MA period throws an explicit 400 Bad Request parameter error.
BACKTEST	/api/v1/backtest	Mutation (POST)	Confirms the generated equity curve array output matches the precise structural formatting required by Recharts components down to the index.

BACKTEST	/api/v1/backtest/stream	Mutation (POST)	Server-Sent Events (SSE) backtest endpoint that delivers equity curve points progressively as they are computed
USER & AGENT SETTINGS	/api/v1/user/settings	Query (GET)	Confirms an authenticated caller receives {found: false} when no agent_settings row exist for that user and {found: true, settings:{...}} when persisted risk parameters/user profile exists.
USER & AGENT SETTINGS	/api/v1/user/settings	Mutation (POST)	Verifies valid agent configuration payload is upserted and returns successfully. Asserts out-of-bounds values eg invalid risk_profile, rebalance_freq fails.
USER & AGENT SETTINGS	/api/v1/user/onboarding/completed	Mutation (POST)	Confirms users.isonboarded is flipped to true and response returns: {status: "onboarding_finalized"}

PORTFOLIO	/api/v1/user/portfolio/summary	Query (GET)	Verifies the latest portfolio_snapshots row for the authenticated user is returned (total asset value, 24h change, P&L fields).
PORTFOLIO	/api/v1/user/portfolio/summary	Query (GET)	Asserts 404 Not Found when no portfolio snapshot history exists for the user.
PORTFOLIO	/api/v1/user/portfolio/history	Query (GET)	Confirms chronological snapshot series is returned; limit query param is clamped (default 30, max 365).
PORTFOLIO	/api/v1/user/portfolio/positions	Query (GET)	Verifies all open positions for the authenticated user's paper account are returned as a JSON array.
PORTFOLIO	/api/v1/user/portfolio/trades	Query (GET)	Confirms recent fills / execution records are returned; limit is respected (default 50, max 500).
PORTFOLIO	/api/v1/user/portfolio/alerts	Query (GET)	Verifies system_alerts for the user are returned ordered by recency; limit clamped (default 10, max 100).

TRADING AGENT	/api/v1/agent/start	Mutation (POST)	Verifies a valid start request spawns an agent worker and returns { status: "activated", symbol, run_id } with 200 OK.
TRADING AGENT	/api/v1/agent/start	Mutation (POST)	Asserts duplicate start for the same user+symbol returns 409 Conflict (agent is already running).
TRADING AGENT	/api/v1/agent/start	Mutation (POST)	Confirms unsupported strategy_type or invalid MA periods return 400 Bad Request.
TRADING AGENT	/api/v1/agent/stop	Mutation (POST)	Verifies stop terminates the worker and returns { status: "terminated", symbol }; updates agent_runs status.
TRADING AGENT	/api/v1/agent/stop	Mutation (POST)	Asserts stop for a non-running symbol returns 404 Not Found.
TRADING AGENT	/api/v1/agent/portfolio/start	Mutation (POST)	Launch a portfolio catalog agent (multi-symbol)

TRADING AGENT	/api/v1/agent/portfolio/stop	Mutation (POST)	Terminate a portfolio agent
TRADING AGENT	/api/v1/agent/list	Query (GET)	List all active agent runs for the authenticated user
TRADING AGENT	/api/v1/strategies/catalog	Query (GET)	Browse the strategy catalog with specs and recommended defaults by risk profile
SYSTEM	/health	Query (GET)	Confirms liveness probe returns 200 OK with { status: "ok", time } for container/orchestration health checks.
PORTFOLIO LIVE	/api/v1/user/portfolio/live	Query (GET, application/x-ndjson stream)	Streams snapshot events and, when Alpaca credentials are available, latest price events for dashboard live marking.
STRATEGY CATALOG	/api/v1/strategies/catalog	Query (GET)	Returns default universe, recommended map, and StrategySpec metadata including risk profile, paper_only, deployment_status,

			benchmark, and evidence paths.
--	--	--	--------------------------------

User Testing

Milestone Planning

Lift off (13 May - 20 May)	
Objective	
Idea finalization and building app foundation	
Deliverables	
Week	Task
1	Set up the basic project workspace and development environments for the team.
1	Create the core database structure to store initial data.
1	Design the company logo and website layout themes.
1	Build the website layout, including the main landing page and dashboard
1	Build a login modal using react portal for seamless authentication
1	Created a static app dashboard with hardcoded static data
Milestone 1 (20 May - 3 June)	
Objective	
Build app skeleton	
Deliverables	
2	Build user login and signup

2	Build back testing engine for simulation of trading strategies
2	Create ma crossover strategy
2	Create 1D Kalman filter strategy
2	Create unit tests for the backend functions
2	Build data ingestion pipeline from Alpaca market data Api
3	Design extensive database schema layout to support new features
3	Developed dashboard, config, portfolio, activity view within the app itself
3	Connect user dashboard to live data feeds instead of static demo placeholders.
3	Design onboarding user flow for new users
3	Create customizable risk profile settings for users
Milestone 2 (3 June - 1 July)	
Objective	
Build the core trading engine	
Deliverables	
4	Improve background system stability to handle multiple concurrent simulation requests safely.
4	Build paper trading engine
4	Build pairs trading engine

4	Improve database structure to store pairings and relationship metrics.
4	Improve the live data connection to track buying and selling pressure
5	Build hidden Markov model
5	Implement safety trigger using HMM
Milestone 3 (1 July - 29 July)	
Objective	
Final implementations and ensuring the app runs smoothly	
Deliverables	
6	Create safety rules for the entire trading system
7	Build a signal processing engine
7	Implement an adjustable weighting framework
7	Run end-to-end simulation tests

Future Roadmap

Monetization Strategy

We consider building revenue through a layered monetization model, introduced in stages according to what is commercially defensible, technically feasible, and legally appropriate at each phase of growth.

1. Freemium SaaS Core

We consider launching the primary revenue model as a freemium SaaS offering. A free tier would give users limited access to backtests, strategies, symbols, and saved results, while a paid Pro tier would unlock expanded compute, full strategy access, live paper-trading agents, saved history, exports, and advanced analytics.

2. Education and B2B2C Licensing

We consider offering O(Alpha) as a teaching and research tool for universities, quant finance clubs, student investment societies, and course creators. The platform can be positioned as a hands-on environment for learning backtesting, Kalman filtering, portfolio construction, and risk metrics.

3. Developer API

We consider introducing a paid, metered Developer API once the platform's backtesting engine and strategy primitives are sufficiently stable. This would allow hobbyist quants, developers, and small research teams to programmatically access core O(Alpha) functionality.

4. Strategy Marketplace

We propose developing a strategy marketplace at a later stage, allowing users to publish, share, or monetize strategy templates through the platform. O(Alpha) could take a revenue share from paid templates or premium strategy access.

Final Reflection

Constraints by Design

O(Alpha) is still a proof of concept, and several of its limits are deliberate scoping decisions rather than unfinished work.

Market data is research-grade, not execution-grade. The platform runs on Alpaca's free IEX feed, which is delayed and covers only a slice of consolidated volume. This is sufficient to study strategy behaviour on daily and intraday bars, but it is not the real-time, full-depth, low-latency data that live execution demands.

Back test model is idealized. Fills occur at the next bar's open with no commission and no slippage, and positions are all-in/all-out. This makes results clean and reproducible for learning, but it omits the transaction costs, partial fills, and market impact that erode real-world performance so reported metrics should be read as upper bounds, not forecasts.

END